

Procédure de maintenance de Passerelle

Application	Passerelle (nom technique <code>onboardcc</code>)
Document	Procédure de maintenance (application déjà installée et en service)
Version	2.0 — 25/09/2026
Public	Exploitant informatique (niveau technique non expert accepté : chaque geste est expliqué)
Document complémentaire	<code>1-procedure_deploiement_passerelle.md</code> — installation initiale ou réinstallation complète

 **Valeurs à remplacer.** Les noms entre chevrons `< >` sont propres à chaque installation : à remplacer par les vraies valeurs **avant** de copier une commande.

Valeur	Signification
<code><COMPTE></code>	Identifiant du compte d'hébergement
<code><HOTE_SSH></code>	Nom d'hôte du serveur pour la connexion SSH
<code><SERVEUR></code>	Nom interne du serveur (affiché dans l'invite du terminal — informatif)
<code><DOMAINE></code>	Adresse de l'application, sans <code>https://</code>
<code><DOSSIER_PUBLIC></code>	Dossier public du domaine, servi par Apache (ex. <code>/home/<COMPTE>/<DOMAINE></code>)
<code><CHEMIN_CLE_SSH></code>	Emplacement de la clé privée SSH sur le poste d'administration

Introduction

Objectif de cette procédure

Une fois Passerelle installée, cette procédure décrit les **gestes que l'exploitant est amené à répéter** pour la garder en bon état : la sauvegarder, y déployer un correctif ou une évolution, diagnostiquer un signalement d'utilisateur, et entretenir sa sécurité.

Chaque étape est **indépendante** : il n'est pas nécessaire de tout lire, seulement l'étape correspondant au besoin.

Quelle étape pour quel besoin ?

Besoin	Étape
Faire une copie de secours de la base et des pièces jointes	1. Sauvegarder
Mettre en service un correctif ou une nouvelle version publiée sur GitHub	2. Déployer une mise à jour

Besoin	Étape
Un utilisateur signale que « ça ne marche pas »	3. Diagnostiquer un problème
Changer un mot de passe de base de données, corriger un certificat HTTPS	4. Sécurité
Consigner ce qui a été livré	5. Journal des livraisons

Comment lire cette procédure

Chaque étape suit le même modèle : « **Avant de commencer** », puis des instructions **numérotées**, chacune indiquant **où** la taper.

Où on tape	Comment le reconnaître	Ce qu'on y tape
PowerShell / terminal du poste	Invite du poste de l'exploitant (ex. PS C:\Users\...)	Uniquement la commande ssh de connexion
bash (terminal du serveur)	Invite qui se termine par \$, ex. [<COMPTE>@<SERVEUR> ~]\$	cd, git, npm, node, psql...
psql (base de données)	Invite qui se termine par =>	Uniquement des requêtes SQL terminées par ; et des commandes commençant par \
cPanel / navigateur	Fenêtre du navigateur	Clics dans l'interface de l'hébergeur

-  **Encadré « ampoule »** : explication complémentaire sur le geste qui précède.
-  **Encadré « attention »** : risque ou erreur fréquente à éviter.
- Chaque bloc de code correspond à **une commande ou une série de commandes à taper ensemble**.

 **Règle pour tout mot de passe créé pour ce projet : uniquement des lettres, des chiffres et les caractères _ - . —** jamais d'autres caractères spéciaux (**# ? & @ : / % ! \$ ' ...**). Voir l'étape 4.1 pour la raison.

Deux gestes communs à plusieurs étapes

Ces deux gestes sont cités partout ci-dessous sous les noms « **Ouvrir une session SSH** » et « **Ouvrir la base** ».

A — Ouvrir une session SSH

(où : *PowerShell ou terminal du poste*)

1. Ouvrir un terminal sur le poste d'administration et taper :

```
ssh -i <CHEMIN_CLE_SSH> <COMPTE>@<HOTE_SSH>
```

2. Saisir la **phrase secrète** de la clé (rien ne s'affiche : c'est normal). L'invite devient `[<COMPTE>@<SERVEUR> ~]$` : **on est dans bash sur le serveur**.
3. Activer l'environnement Node du projet et se placer dans le dossier du code :

```
source /home/<COMPTE>/nodeenv/passerelle-api/24/bin/activate && cd ~/passerelle-api
```

Résultat attendu : (24) apparaît au début de l'invite.

💡 L'activation ne dure que le temps de la session : à refaire à chaque nouvelle connexion, sinon `pnpm: command not found`. Le `cd ~/passerelle-api` est indispensable : `git` ne fonctionne qu'à l'intérieur du dossier du code (sinon `fatal: not a git repository`).

⚠ **Connection timed out** : l'adresse IP du poste n'est pas (ou plus) autorisée. Voir la procédure d'installation, étape 1.4.

B — Ouvrir la base

(où : *bash*, sur le serveur — session *SSH* ouverte)

```
psql -h localhost -U <COMPTE>_passerelle_app -d <COMPTE>_passerelle
```

Saisir le mot de passe de la base (rien ne s'affiche). L'invite devient `<COMPTE>_passerelle=>` : **on est dans psql**. Pour en sortir et revenir dans *bash* :

```
\q
```

Étape 1 — Sauvegarder l'application

Ce que fait cette étape : produire une copie de secours de tout ce qui n'est pas dans le code : la **base de données** (comptes, candidats, postes, opportunités...) et le dossier des **pièces jointes** (`uploads/`). Le code n'a pas besoin d'être sauvegardé : il est sur GitHub.

Avant de commencer :

- ☐ Une session *SSH* ouverte (geste A).
- ☐ Un endroit de stockage **hors du serveur** pour conserver les copies (poste de l'exploitant, disque réseau de l'organisation...). Une sauvegarde qui reste sur le serveur ne protège pas d'une panne du serveur.

💡 **Quand la faire ?** Avant tout déploiement de mise à jour (étape 2), avant tout nettoyage de la base, et régulièrement (recommandation : **chaque semaine** en production, davantage à l'approche de périodes de forte activité).

1.1 Sauvegarder la base de données

(où : *bash*, sur le serveur)

```
mkdir -p ~/sauvegardes  
pg_dump -h localhost -U <COMPTE>_passerelle_app -d <COMPTE>_passerelle -F c -f  
~/sauvegardes/base_$(date +%Y%m%d).dump
```

Saisir le mot de passe de la base. Résultat attendu : la commande se termine sans message, et le fichier existe :

```
ls -lh ~/sauvegardes/
```

💡 -F c produit un fichier compressé au format PostgreSQL. \$(date +%Y%m%d) ajoute la date du jour au nom (base_20260925.dump) : une sauvegarde par jour ne s'écrase pas.

1.2 Sauvegarder les pièces jointes

(où : *bash*, sur le serveur)

```
tar -czf ~/sauvegardes/uploads_$(date +%Y%m%d).tar.gz -C ~/passerelle-api uploads
```

Résultat attendu : aucun message, et un fichier uploads_AAAAMMJJ.tar.gz dans ~/sauvegardes/.

1.3 Copier les sauvegardes hors du serveur

(où : *PowerShell*, sur le poste — ***pas*** dans la session *SSH* : ouvrir une nouvelle fenêtre)

```
scp -i <CHEMIN_CLE_SSH> "<COMPTE>@<HOTE_SSH>:~/sauvegardes/*"  
"C:\Sauvegardes_Passerelle\"
```

Résultat attendu : les fichiers .dump et .tar.gz apparaissent dans le dossier du poste (à créer avant si besoin).

⚠ Les sauvegardes contiennent des **données personnelles** (RGPD) : les conserver dans un emplacement à accès restreint et **supprimer les anciennes** (recommandation : garder les 4 dernières semaines).

1.4 Restaurer une sauvegarde de la base (en cas de besoin)

(où : *bash*, sur le serveur — après avoir déposé le fichier .dump dans ~/sauvegardes/)

⚠ La restauration **remplace** le contenu actuel de la base. Ne la faire qu'en cas de perte de données, après avoir fait une sauvegarde de l'état actuel (1.1) au cas où.

```
pg_restore -h localhost -U <COMPTE>_passerelle_app -d <COMPTE>_passerelle --clean
--if-exists --no-owner ~/sauvegardes/base_AAAAMMJJ.dump
```

💡 Cette procédure de sauvegarde et de restauration est à **rejouer sur un environnement de test** avant la mise en production, pour s'assurer qu'une sauvegarde est bien restaurable.

Étape 2 — Déployer une mise à jour

Ce que fait cette étape : mettre en service sur le serveur un correctif ou une évolution qui a été publié sur le dépôt GitHub <https://github.com/DamienMoreauConsultant/onboardcc> (branche **main**).

💡 **Principe à retenir : le serveur ne fait que récupérer.** Le code n'est modifié que sur le poste du développeur puis envoyé à GitHub. Le serveur fait uniquement **git pull** (récupérer) et ne fait jamais **git add**, **git commit** ni **git push**. Ainsi, rien ne peut écraser une version par erreur.

Avant de commencer :

- ☐ Le développeur confirme que la version à déployer est **poussée sur GitHub** : ouvrir la page du dépôt et vérifier que le dernier commit (message et date) est le bon. Le serveur ne récupère que ce qui est sur GitHub.
- ☐ Une **sauvegarde récente** existe (étape 1), surtout si la mise à jour comprend une migration de base (2.7).
- ☐ La nature de la mise à jour est connue (backend, interface, base de données, variables) : la liste fournie avec la livraison indique quelles sous-étapes sont à faire. Dans le doute, faire **2.1 à 2.6** dans l'ordre.

La mise à jour modifie...	Sous-étapes à faire
Uniquement l'interface (artifacts/onboardcc/)	2.1, 2.2, 2.5, 2.8
Le backend (artifacts/api-server/)	2.1, 2.2, 2.4, 2.8 (+ 2.3 si les dépendances changent)
La base de données (fichier migrations/0XX_...sql)	2.1, 2.2, 2.7 avant 2.4, puis 2.8
Une variable d'environnement	2.6 (puis 2.8)

2.1 Ouvrir une session SSH

Suivre le **geste A** de l'introduction (connexion, activation de l'environnement, dossier du code).

2.2 Récupérer le nouveau code

(où : *bash*, sur le serveur)

1 — Vérifier que le serveur n'a aucune modification locale

```
git status
```

Résultat attendu : **nothing to commit, working tree clean**. Si des fichiers sont listés comme modifiés, s'arrêter et lire l'encadré ci-dessous avant de continuer.

2 — Récupérer les nouveautés

```
git pull
```

Résultat attendu : la liste des fichiers modifiés s'affiche (ex. **6 files changed, 64 insertions(+), 30 deletions(-)**). Si le message est **Already up to date.**, il n'y a rien de nouveau : vérifier auprès du développeur que l'envoi vers GitHub a bien été fait.

⚠ Si git pull échoue avec **Your local changes to the following files would be overwritten by merge**, le serveur contient une modification locale. **Ne jamais faire git add ni git commit sur le serveur.** À la place :

1. Regarder ce qui diffère : **git diff <fichier>**.
2. Si la modification est un effet de bord d'un outil (cas connu : **pnpm-workspace.yaml** modifié par **pnpm approve-builds**) : l'annuler avec **git restore <fichier>**, puis relancer **git pull**.
3. Si la modification semble volontaire (quelqu'un a corrigé quelque chose directement sur le serveur) : **ne pas trancher seul**. Prévenir le développeur pour qu'il reporte cette correction dans le code, l'envoie sur GitHub, puis relancer **git pull**.

2.3 Mettre à jour les dépendances (si elles ont changé)

(où : *bash*, sur le serveur)

À faire seulement si **git pull** a listé **package.json**, **pnpm-lock.yaml** ou **pnpm-workspace.yaml** parmi les fichiers modifiés. Dans le doute, le lancer : il répond **Already up to date** si rien n'a changé.

```
pnpm install
```

⚠ Un **pnpm: command not found** signifie que l'environnement Node n'est pas activé (l'invite ne commence pas par **(24)**) : refaire l'instruction 3 du geste A. Si un script est signalé comme bloqué, lancer **pnpm approve-builds** puis un nouveau **pnpm install**, et contrôler **git status** ensuite (voir 2.2).

2.4 Compiler et redémarrer le backend

(où : *bash*, puis *cPanel*)

À faire si le backend (**artifacts/api-server/**) a changé, ou si un doute subsiste.

1 — Compiler

```
cd ~/passerelle-api/artifacts/api-server
pnpm run build
```

Résultat attendu : une liste de fichiers `dist/...` puis ⚡ Done.

2 — Redémarrer l'application

(où : *cPanel*) *cPanel* → **Setup Node.js App** → l'application → bouton **Redémarrer** (« RESTART »).

💡 Le serveur exécute le fichier compilé `dist/index.mjs`, pas le code source : sans compilation, l'ancien code continue de tourner ; sans redémarrage, le nouveau code compilé n'est pas chargé.

2.5 Compiler et publier l'interface

(où : *bash*, sur le serveur)

À faire si l'interface (`artifacts/onboardcc/`) a changé.

1 — Compiler

```
cd ~/passerelle-api/artifacts/onboardcc
export PORT=5000
export BASE_PATH=/
export VITE_API_BASE_URL=/api
pnpm run build
```

⚠ Les trois valeurs `PORT`, `BASE_PATH` et `VITE_API_BASE_URL` doivent être **identiques à l'installation initiale** : une valeur différente casserait le chargement de l'interface.

2 — Publier les fichiers compilés

```
cp -r dist/public/* <DOSSIER_PUBLIC>/
```

💡 Le dossier public n'est **jamais** lu directement depuis le code : sans cette copie, `git pull` seul ne change rien à ce que voient les visiteurs. L'interface est un ensemble de fichiers statiques servis par Apache : **aucun redémarrage** n'est nécessaire.

2.6 Ajouter ou modifier une variable d'environnement

(où : *bash*, puis *cPanel*)

À faire si la livraison introduit ou change une variable (ex. nouveau paramètre de barème `MATCH_SCORE_*`).

1 — Consulter la liste de référence à jour

```
cat ~/passerelle-api/artifacts/api-server/.env.example
```

2 — Saisir la variable

(où : *cPanel*) cPanel → **Setup Node.js App** → l'application → **Environment variables** :

- variable **nouvelle** : bouton **Ajouter une variable**, nom exact et valeur (voir `.env.example`) ;
- variable **existante** : icône crayon sur sa ligne.

Puis **Enregistrer**.

💡 L'application redémarre **automatiquement** à l'enregistrement : pas de bouton « Redémarrer » à cliquer pour ce cas précis.

⚠ **Aucune variable `MATCH_SCORE_*` ne doit manquer** : le calcul du score les vérifie toutes d'un coup, et une seule manquante ou invalide fait échouer **tous** les calculs, avec l'erreur `Configuration de scoring absente ou invalide : NOM_DE_LA_VARIABLE`. Comparer soigneusement `.env.example` avec cPanel.

2.7 Appliquer une migration de base de données

(où : *bash*, puis *psql*)

À faire si la livraison contient un fichier `artifacts/api-server/migrations/0XX_nom.sql`. À faire **après** `git pull` (2.2) et **avant** de redémarrer le backend (2.4) : le nouveau code peut dépendre du nouveau schéma.

Le fichier est déjà sur le serveur : `git pull` l'a récupéré.

💡 Une migration ne modifie que la **base** (structure ou données), jamais le code, et n'installe jamais de paquet. Les bibliothèques nécessaires au code sont installées par `pnpm install` (2.3).

1 — Lire l'en-tête du fichier (dans *bash*)

```
head -30 ~/passerelle-api/artifacts/api-server/migrations/0XX_nom.sql
```

Les commentaires en tête expliquent ce que fait la migration et si une action est à faire avant ou après.

2 — Ouvrir la base (dans *bash* → *psql*) : geste B de l'introduction.

3 — Charger la migration (dans *psql*)

```
\i artifacts/api-server/migrations/0XX_nom.sql
```

Résultat attendu : aucune ligne **ERREUR**. Si le fichier ne se termine pas par un `COMMIT;`, le taper soi-même après avoir lu le résultat.

4 — Quitter psql (dans *psql*)

\q

5 — Vérifier le schéma (dans bash)

```
cd ~/passerelle-api/artifacts/api-server
pnpm run check-seed
cd ~/passerelle-api
```

Le nombre de tables attendu est confirmé.

⚠ Pour toute requête manuelle contenant des **accents**, voir 4.4 avant de taper quoi que ce soit dans psql.

2.8 Vérifier que tout fonctionne

À dérouler **après tout déploiement**, avant de le considérer terminé. N'utiliser que des données de test.

#	Test	Comment	Attendu
1	Le serveur répond	Dans bash : <code>curl -s https://<DOMAINE>/api/healthz</code>	<code>{"status":"ok"}</code>
2	L'interface se charge	Ouvrir <code>https://<DOMAINE>/</code> puis Ctrl+F5	L'écran de connexion s'affiche, sans erreur dans la console (touche F12)
3	La connexion fonctionne	Se connecter avec un compte de test	Accès à l'écran attendu selon le rôle
4	La zone modifiée fonctionne	Selon le correctif livré	Comportement conforme à la demande
5	Rien d'autre n'est cassé	Parcourir 2 ou 3 écrans sans rapport avec le correctif	Aucune régression

💡 Après une évolution **majeure**, faire la recette complète de la procédure d'installation (étape 10.1).

Étape 3 — Diagnostiquer un problème

Ce que fait cette étape : trouver rapidement pourquoi un utilisateur signale que « ça ne marche pas », des vérifications les plus rapides aux plus poussées.

Avant de commencer :

- ☐ Le **symptôme précis** : quel écran, quel message affiché, à quelle heure, quel compte.
- ☐ Une session SSH ouverte (geste A).

3.1 Vérifier que le serveur répond

(où : *bash*, sur le serveur)

```
curl -s https://<DOMAINE>/api/healthz
```

Résultat attendu : `{"status":"ok"}`.

💡 `curl` envoie une requête web depuis le terminal comme le ferait un navigateur, et affiche la réponse brute. C'est plus rapide qu'un navigateur et cela ne dépend pas du certificat (si `curl` répond `SSL certificate problem`, voir 3.3).

- Réponse `{"status":"ok"}` → le serveur fonctionne : le problème est ailleurs (compte, données, navigateur de l'utilisateur). Demander un `Ctrl+F5` et le message exact.
- Pas de réponse ou autre réponse → passer au 3.2.

3.2 Redémarrer l'application, puis lire l'erreur réelle

1 — Redémarrer

(où : *cPanel*) *cPanel* → **Setup Node.js App** → l'application → **Redémarrer**, puis refaire le test 3.1.

2 — Si cela ne suffit pas : consulter les erreurs

(où : *cPanel*) barre de recherche → **Erreurs**.

3 — Si le message n'est pas exploitable : lancer le serveur à la main

(où : *bash*, sur le serveur)

Sur un hébergement mutualisé, l'erreur réelle de Node.js est souvent invisible. Reproduire le démarrage à la main l'affiche (guillemets **simples** obligatoires autour des valeurs) :

```
cd ~/passerelle-api
export PORT=3000
export DATABASE_URL='postgresql://...'
export JWT_SECRET='...'
node artifacts/api-server/dist/index.mjs
```

Reprendre les valeurs affichées dans *cPanel* → **Environment variables**. Le message obtenu indique la cause (`PORT` manquante, variable de score absente, connexion à la base refusée...). Arrêter avec `Ctrl+C`, corriger la variable dans *cPanel*, puis redémarrer l'application.

Message	Cause	Que faire
<code>PORT environment variable is required</code>	Variable <code>PORT</code> absente	Ajouter <code>PORT=3000</code> dans <i>cPanel</i> (2.6)
<code>Invalid URL</code>	Caractère spécial dans le mot de passe de <code>DATABASE_URL</code>	Changer le mot de passe (4.2)

Message	Cause	Que faire
Configuration de scoring absente ou invalide	Variable <code>MATCH_SCORE_*</code> absente	2.6
authentification par mot de passe échouée	Mot de passe de <code>DATABASE_URL</code> différent de celui de la base	4.2

3.3 Vérifier le certificat HTTPS

Symptôme : le navigateur affiche « Votre connexion n'est pas privée »

(`NET::ERR_CERT_AUTHORITY_INVALID`), ou `curl` répond `SSL certificate problem: self signed certificate`.

Vérifier : (où : *cPanel*) barre de recherche → **SSL/TLS Status** → repérer la ligne du domaine. Si elle affiche **Self-signed**, la correction est décrite en **4.3**.

Étape 4 — Sécurité

Ce que fait cette étape : entretenir la sécurité : mots de passe, certificat, précautions d'encodage.

4.1 Règle des mots de passe

Pour **tout** mot de passe créé pour ce projet (base, compte email, comptes applicatifs) : uniquement des lettres, des chiffres et les caractères `_ - .`.

 **Pourquoi une règle aussi stricte ?** Des caractères spéciaux classiques ont provoqué des échecs difficiles à comprendre : `!` est interprété par le terminal comme un rappel d'historique ; `#`, `?` et `&` ont un sens réservé dans une adresse et cassent la lecture de `DATABASE_URL` (message générique `Invalid URL`, très difficile à relier à sa vraie cause). Seuls `_ - .` sont sûrs partout (terminal, adresse, SQL). La longueur compte bien plus que la variété des symboles.

 **Comptes applicatifs :** la même règle est **exigée par l'application** pour les comptes candidat, recruteur, chargé de mission et administrateur : **12 caractères minimum**, avec au moins une minuscule, une majuscule, un chiffre et l'un des trois caractères `_ - .`. Aucun autre caractère spécial n'est accepté.

4.2 Changer le mot de passe de la base de données

Avant de commencer :

- ☐ Le nouveau mot de passe choisi (règle 4.1).
- ☐ Une session SSH ouverte (geste A).

1 — Ouvrir la base (*dans bash* → *psql*) : geste B de l'introduction.

2 — Changer le mot de passe côté PostgreSQL (*dans psql*)

```
ALTER USER <COMPTE>_passerelle_app WITH PASSWORD 'NOUVEAU_MOT_DE_PASSE';
```

Résultat attendu : **ALTER ROLE**. Quitter psql :

```
\q
```

3 — Mettre à jour **DATABASE_URL** (dans cPanel)

cPanel → **Setup Node.js App** → l'application → **Environment variables** → icône crayon sur **DATABASE_URL** → ne remplacer que le mot de passe dans :

```
postgresql://<COMPTE>_passerelle_app:NOUVEAU_MOT_DE_PASSE@localhost:5432/<COMPTE>_passerelle
```

→ **Enregistrer**. L'application redémarre automatiquement.

4 — Vérifier (dans bash)

```
curl -s https://<DOMAINE>/api/healthz
```

Puis se connecter à l'application dans le navigateur.

4.3 Corriger un certificat HTTPS auto-signé

Avant de commencer :

- ☐ Le diagnostic 3.3 confirme que le certificat du domaine est **Self-signed**.

(où : cPanel)

- Barre de recherche → **Let's Encrypt** → outil **Let's Encrypt™ SSL**.
- Section **Générer un nouveau certificat** → rechercher le domaine.
- Cliquer sur **Générer**, garder les options par défaut (les deux domaines cochés, méthode HTTP-01, clé RSA 2048 bits).
- Cliquer sur **Générer** (et non « Simuler la génération »).
- Attendre quelques secondes (Apache redémarre en arrière-plan), puis rouvrir <https://<DOMAINE>/>.

💡 Un nouveau domaine reçoit d'abord un certificat auto-signé temporaire. Le système automatique (**AutoSSL**) **ignore volontairement** tout domaine qui a déjà un certificat non émis par lui : sans cette intervention, le problème ne se corrige jamais seul. Ce piège concerne **tout nouveau domaine** sur ce type d'hébergement, y compris le futur serveur de production s'il utilise cPanel.

4.4 Éviter la corruption des accents dans la base

Le risque : sur un poste Windows, `psql` peut mal interpréter les caractères accentués (é, è, à, ç...) si l'encodage de la console diffère de celui de la base (UTF-8 partout dans ce projet). Le danger est **silencieux** : un accent tapé au clavier dans une requête peut être mal encodé et corrompre une donnée sans aucun message d'erreur, même quand aucun avertissement n'est affiché (notamment en session SSH).

Règle 1 — ne jamais taper de texte accentué directement dans psql. Préparer un fichier `.sql`, l'enregistrer en **UTF-8** (le Bloc-notes Windows propose parfois « ANSI » : à éviter), puis le charger (*dans psql*) :

```
\i chemin/vers/le_fichier.sql
```

Règle 2 — si une requête manuelle est vraiment inévitable (ex. un `SELECT` de diagnostic), depuis un poste Windows, forcer l'encodage **avant** de se connecter (*dans PowerShell*) :

```
$env:PGCLIENTENCODING = "UTF8"
```

💡 Cette commande ne dure que le temps de la fenêtre PowerShell : à refaire à chaque nouvelle fenêtre. Si l'avertissement d'encodage persiste, taper `chcp 65001` dans la même fenêtre **avant** toute connexion.

4.5 Réinitialiser le mot de passe PostgreSQL d'un poste de développement

⚠ **Hors serveur de production** : ne concerne que le PostgreSQL installé sur le poste d'un développeur (quand le mot de passe du compte `postgres` est perdu). Voir [procédure_local_environnement_dev.md](#).

Étape 5 — Journal des livraisons

Ce que fait cette étape : garder la trace de **chaque mise à jour réellement déployée** : quoi, quand, avec quelles actions. En cas de souci après une livraison, ce journal indique immédiatement ce qui a changé.

Avant de commencer :

- ☐ La mise à jour est déployée et vérifiée (étape 2, y compris 2.8).

Comment faire : copier le modèle ci-dessous à la suite du journal, **une entrée par livraison**, dans l'ordre chronologique. **Ne jamais réécrire une entrée déjà renseignée** : uniquement en ajouter.

```
### JJ/MM/AAAA — <Test ou Production> — <objet court>

**Objet** : <une phrase : correctif suite à un signalement / évolution
fonctionnelle / montée de version>

**Contenu** :
- <fichiers, dossiers ou numéros de retour couverts>

**Actions réalisées** (cocher) :
```

- [] Sauvegarde préalable (étape 1)
- [] Code récupéré (2.2) + backend (2.4) et/ou interface (2.5)
- [] Dépendances (2.3)
- [] Migration(s) SQL (2.7) – fichier(s) : <nom(s)>
- [] Variable(s) d'environnement (2.6) – variable(s) : <nom(s)>
- [] Vérifications (2.8) : OK

****Résultat**** : <OK / problème rencontré et sa résolution>

Livraisons enregistrées

(aucune à ce jour — la première entrée sera créée lors de la mise en production)

Historique des versions

Version	Date	Modification
1.x	15/09/2026 – 25/09/2026	Création (scission de l'ancienne annexe « Maintenance courante ») et ajouts successifs.
2.0	25/09/2026	Refonte complète en livrable : introduction (objectif, choix de l'étape selon le besoin, conventions, deux gestes communs), une étape par thème (« Avant de commencer », instructions numérotées, terminal précisé pour chaque commande). Étape 1 « Sauvegarder » rédigée (sauvegarde et restauration de la base, sauvegarde des pièces jointes, copie hors serveur).